Unit I- Introduction to Data Structures and Algorithms

Introduction: Introduction to Data Structures: Abstract Data Types (ADT), Linear and Non-linear, Static and Dynamic, Persistent and Ephemeral data structures
Algorithms: Space complexity, Time complexity, Asymptotic notation- Big-O, Theta and Omega,finding complexity using step count method, Analysis of programming constructs-Linear, Quadratic,Cubic, Logarithmic.
Algorithmic Strategies: Introduction to algorithm design strategies- Divide and Conquer, and Greedy strategy

Introduction to Data Structures:

Data Structures are specialized formats for organizing, storing, and managing data efficiently in a computer system. They provide a way to structure data so that it can be used effectively for different types of operations such as insertion, deletion, traversal, searching, and sorting. Common data structures include arrays, linked lists, stacks, queues, trees, and graphs. Each structure is chosen based on the specific needs of an application, such as speed, memory usage, and ease of implementation. Data structures are fundamental to programming and algorithm design, as they directly impact the performance and scalability of software systems. Understanding data structures allows developers to write optimized code, solve complex problems efficiently, and build robust applications.

Abstract Data Types (ADT) are theoretical concepts in computer science that define a data structure purely by the operations that can be performed on it and the rules those operations must follow—without specifying how these operations are implemented. ADTs focus on what the data structure does, not how it does it. Common examples include List, Stack, Queue, Deque, Tree, and Graph. For instance, a Stack ADT supports operations like push, pop, and peek based on the Last In, First Out (LIFO) principle, regardless of whether it's implemented using an array or a linked list. ADTs promote modularity, abstraction, and reusability in software design, allowing programmers to separate logic from implementation details.

Linear data structures are data structures in which data elements are arranged sequentially or linearly, where each element is connected to its previous and next element. Examples

Click Here join Telegram channel

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

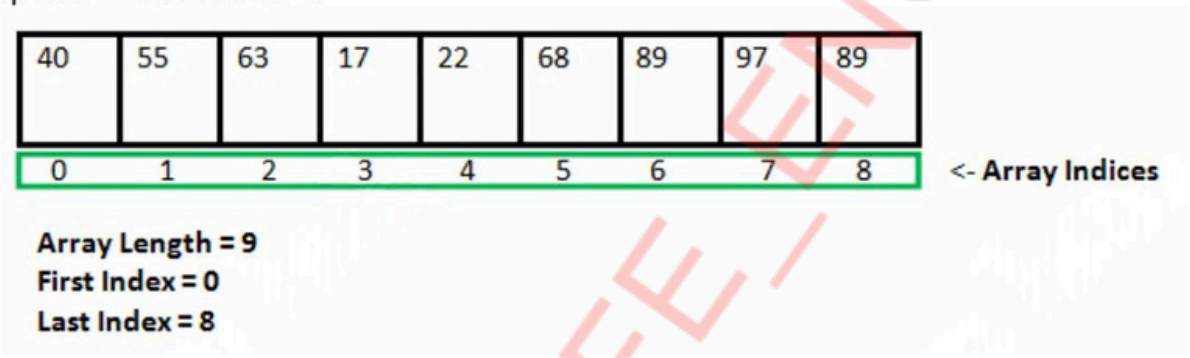
© 2026 Vishwajeet Londhe. All Rights Reserved.

include arrays, linked lists, stacks, and queues. In these structures, traversal is done in a single level

from one element to another in a linear manner. On the other hand, **non-linear data structures** are those in which data elements are not arranged in a sequential manner. Instead, they are organized in a hierarchical or interconnected way, allowing multiple relationships between elements. Common examples are trees and graphs, where elements may have one-to-many or many-to-many relationships. Linear structures are easier to implement and use less complex logic, while non-linear structures are more efficient for representing complex relationships.

What is a Static Data structure?

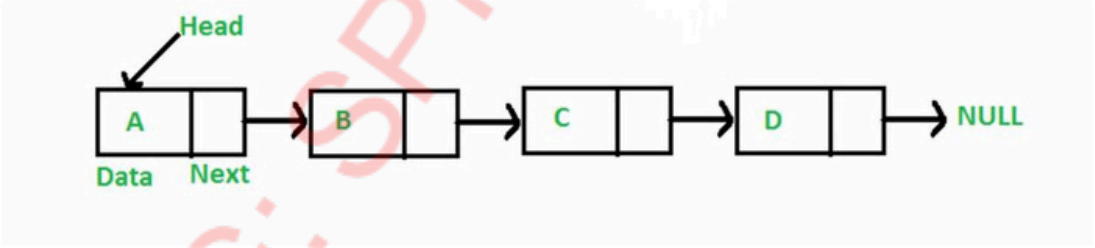
In Static data structure the size of the structure is fixed. The content of the data structure can be modified but without changing the memory space allocated to it.



Example of Static Data Structures: Array

What is Dynamic Data Structure?

In Dynamic data structure the size of the structure is not fixed and can be modified during the operations performed on it. Dynamic data structures are designed to facilitate change of data structures in the run time.



Static Data Structure vs Dynamic Data Structure

- Static data structures, such as arrays, have a fixed size and are allocated at compile-time. This means that their memory size cannot be changed during program execution. Index-based access to elements is fast and efficient since the address of the element is known.
- Dynamic data structures, on the other hand, have a variable size and are allocated at run-time. This means that their memory size can be changed during program execution. Memory can be dynamically allocated or deallocated during program execution. Due to this dynamic

TG: SPPU_FE_ENGG

Click Here join Telegram channel

nature, accessing elements based on index may be slower as it may require memory allocation and deallocation.

Aspect	Static Data Structure	Dynamic Data Structure
Memory allocation	Memory is allocated at compile-time	Memory is allocated at run-time
Size	Size is fixed and cannot be modified	Size can be modified during runtime
Memory utilization	Memory utilization may be inefficient	Memory utilization is efficient as memory can be reused
Access	Access time is faster as it is fixed	Access time may be slower due to indexing and pointer usage
Examples	Arrays, Stacks, Queues, Trees (with fixed size)	Lists, Trees (with variable size), Hash tables

TG: SPPU_FE_ENGG

Persistent vs. Ephemeral

- An *ephemeral* data structure is one for which only one version is available at a time: after an update operation, the structure as it existed before the update is lost.
- A *persistent* structure is one where multiple versions are simultaneously accessible: after an update, both old and new versions can be used.

Time Complexity: The time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input. Note that the time to run is a

Click Here join Telegram channel

function of the length of the input and not the actual execution time of the machine on which the algorithm is running on.

The valid algorithm takes a finite amount of time for execution. The time required by the algorithm to solve given problem is called time complexity of the algorithm. Time complexity is very useful measure in algorithm analysis.

It is the time needed for the completion of an algorithm. To estimate the time complexity, we need to consider the cost of each fundamental instruction and the number of times the instruction is executed.

Space Complexity:
Definition -

Problem-solving using computer requires memory to hold temporary data or final result while the program is in execution. The amount of memory required by the algorithm to solve given problem is called space complexity of the algorithm.

The space complexity of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input. Consider an example: Suppose a problem to find the frequency of array elements.

It is the amount of memory needed for the completion of an algorithm.

To estimate the memory requirement we need to focus on two parts:

(1) A fixed part: It is independent of the input size. It includes memory for instructions (code), constants, variables, etc.

(2) A variable part: It is dependent on the input size. It includes memory for recursion stack, referenced variables, etc.

Asymptotic Notations:

Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.
These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.
Rather than comparing algorithms directly, asymptotic analysis focuses on understanding the relative growth rates of algorithms' complexities.
It enables comparisons of algorithms' efficiency by abstracting away machine-specific constants and implementation details, focusing instead on fundamental trends.
Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.
By using asymptotic notations, such as Big O, Big Omega, and Big Theta, we can categorize algorithms based on their worst-case, best-case, or average-case time or space complexities, providing valuable insights into their efficiency.

TG: SPPU_FE_ENGG

Click Here join Telegram channel

There are mainly three asymptotic notations:

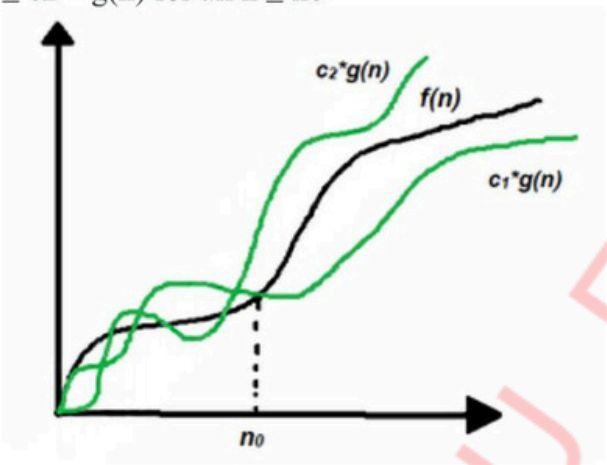
Big-O Notation (O-notation)
Omega Notation (Ω-notation)
Theta Notation (Θ-notation)

1. Theta Notation (Θ-Notation):

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the **average-case** complexity of an algorithm.

.Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Θ(g), if there are constants c1, c2 > 0 and a natural number n0 such that c1* g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0



2. Big-O Notation (O-notation):

Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.

.It is the most widely used notation for Asymptotic analysis.

.It specifies the upper bound of a function.

.The maximum time required by an algorithm or the worst-case time complexity.

.It returns the highest possible output value(big-O) for a given input.

.Big-O(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

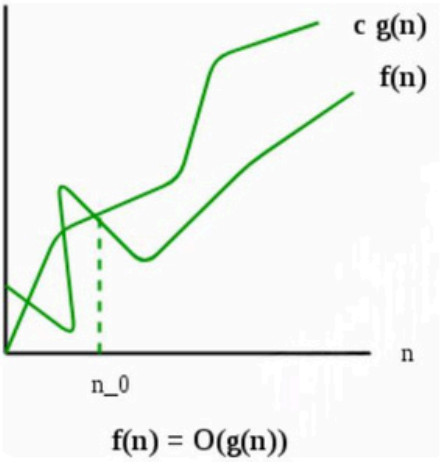
If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that, 0 ≤ f(n) ≤ cg(n) for all n ≥ n0

It returns the highest possible output value (big-O)for a given input.

The execution time serves as an upper bound on the algorithm's time complexity.

TG SPPU_FE_ENGG

Click Here join Telegram channel



f(n) = O(g(n))

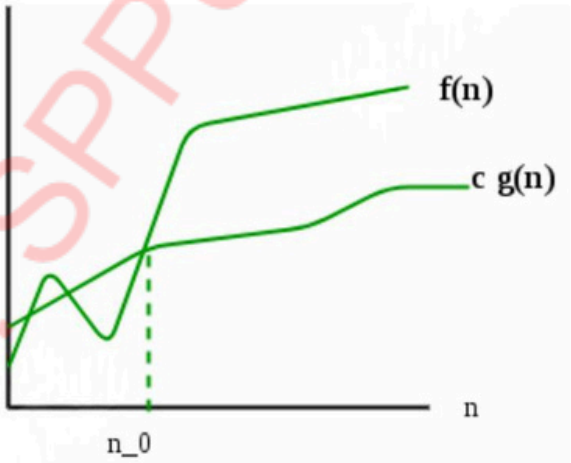
3. Omega Notation (Ω-Notation):

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

The execution time serves as a lower bound on the algorithm's time complexity.

It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Ω(g), if there is a constant c > 0 and a natural number n0 such that c*g(n) ≤ f(n) for all n ≥ n0



f(n) = Omega(g(n))

What is Step Count Method?

TG: SPPU_FE_ENGG

Click Here join Telegram channel

The step count method is one of the methods to analyze the Time complexity of an algorithm. In this method, we count the number of times each instruction is executed. Based on that we will calculate the Time Complexity.

The step Count method is also called as Frequency Count method.

1. Comments:
Comments are used for giving extra meaning to the program. They are not executed during the execution. Comments are ignored during execution.
Therefore the number of times that a comment executes is 0.
2. Conditional statements:
Conditional statements check the condition and if the condition is correct then the conditional subpart will be executed. So the execution of conditional statements happens only once. The compiler will execute the conditional statements to check whether the condition is correct or not so it will be executed one time.

In if-else statements the if statement is executed one time but the else statement will be executed zero or one time because if the "if" statement is executed then the else statement will not execute.
In switch case statements the starting switch(condition) statement will be executed one time but the inner case statements will execute if none of the previous case statements are executed.
In nested if and if else ladder statements also the initial if statement is executed at least once but inner statements will be executed based on the previous statements' execution.
3. Loop statements:
Loop statements are iterative statements. They are executed one or more times based on a given condition.

A typical for($i = 0; i \leq n; i++$) statement will be executed "n+1" times for the first n times the condition is satisfied and the inner loop will be executed and for the (n+1)th time the condition is failed and the loop terminates.
While: The statement is executed until the given condition is satisfied.
Do while: The statement will repeat until the given condition is satisfied. The do-while statement will execute at least once because for the first time it will not check the condition.
4. Functions:
Functions are executed based on the number of times they get called. If they get called n times they will be executed n times. If they are not called at least once then they will not be executed. Other statements like BEGIN, END and goto statements will be executed one time.

Analysis of Linear Search algorithm

Let us consider a Linear Search Algorithm.

```
Linearsearch(arr, n, key)
{
   i = 0;
   for(i = 0; i < n; i++)
   {
      if(arr[i] == key)
```

TG_SPRU_FE_ENGG

Click Here join Telegram channel

```
    {
        printf("Found");
    }
}
```

Where,

- **i = 0**, is an initialization statement and takes O(1) times.
- **for(i = 0;i < n ; i++)**, is a loop and it takes O(n+1) times .
- **if(arr[i] == key)**, is a conditional statement and takes O(1) times.
- **printf("Found")**, is a function and that takes O(0)/O(1) times.

Therefore Total Number of times it is executed is **n + 4** times. As we ignore lower exponents in time complexity total time became **O(n)**.
Time complexity: O(n).
Auxiliary Space: O(1)

Analysis of programming constructs- Linear, Quadratic, Cubic, Logarithmic.

In algorithm analysis, we evaluate how the **execution time** of a block of code grows with the **input size n**. This helps determine the **efficiency** of algorithms in real-world scenarios.

1. Linear Time – O(n):

- Time increases **proportionally** with the input size.
- Common in **single loops** or **simple traversal**.

📌 Example:

```
cpp
CopyEdit
for(int i = 0; i < n; i++) {
   cout << i;
}
```

2. Quadratic Time – O(n²):

- Time increases **with the square** of input size.
- Typically found in **nested loops** over the same data.

📌 Example:

```
cpp
CopyEdit
for(int i = 0; i < n; i++) {
```

TG: SPPU_FE_ENGG

Click Here join Telegram channel

```
    for(int j = 0; j < n; j++) {
        cout << i + j;
    }
}
```

3. Cubic Time – $O(n^3)$:

- Time grows **with the cube** of input size.
- Seen in **three-level nested loops**.

📌 **Example:**

```
cpp
CopyEdit
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        for(int k = 0; k < n; k++) {
            cout << i + j + k;
        }
    }
}
```

4. Logarithmic Time – O(log n):

- Time grows **very slowly** compared to input size.
- Common in **binary search**, divide-and-conquer approaches.

📌 **Example:**

```
cpp
CopyEdit
int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == x)
            return mid;
        else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

TG: SPPU_FE_ENGG

Introduction to Algorithm Design Strategies

Algorithm design strategies are **techniques used to solve problems efficiently** by breaking them down into manageable parts or making optimal decisions. Two widely used strategies are **Divide and Conquer** and **Greedy Strategy**.

1. Divide and Conquer Strategy:

Click Here join Telegram channel

The **Divide and Conquer** approach solves a problem by:

1. **Dividing** the problem into smaller subproblems.
2. **Conquering** the subproblems by solving them recursively.
3. **Combining** the results to form the final solution.

It is widely used in recursive algorithms and is efficient for large data sets.

📌 **Examples:**

- Merge Sort
- Quick Sort
- Binary Search
- Strassen's Matrix Multiplication

🔹 2. Greedy Strategy:

The **Greedy approach** solves a problem by:

1. Making **the best possible (local optimal) choice** at each step,
2. Hoping that these local choices will lead to a **globally optimal solution**.

It doesn't backtrack and is simpler and faster, but not always guaranteed to give the best solution for every problem.

📌 **Examples:**

- Fractional Knapsack Problem
- Prim's Algorithm (for MST)
- Kruskal's Algorithm (for MST)
- Huffman Coding

TG: SPPU_FE_ENGG

Click Here join Telegram channel